Data Example: Poverty in WI and IL

Spatial Regression Models

Mahedi Hasan and Jun Zhu

Exploratory Data Analysis

In this illustration, we will explore American Community Survey data in Wisconsin and Illinois.

We first load the libraries we will be using, `spdep`, `spatialreg`, `tidyr`, `ggplot2`, `gridExtra`, `rgdal`, and `MASS`.

```
library(spdep)
library(spatialreg)
library(tidyr)
library(ggplot2)
library(gridExtra)
library(sf) # replaced for rgdal
library(MASS)
```

Background

The American Community Survey is an ongoing statistical survey conducted by the U.S. Census Bureau and the data are publicly available.

Our research objectives are to quantify the spatial pattern of county-level poverty rates in Wisconsin and Illinois and to assess the relationship between county-level poverty rates and several socio-economic factors.

We focus on data for the census year 2000 in Wisconsin and Illinois. The original data at the individual level are aggregated to the county level. The counties are identified by the 5-digit federal information processing (FIPS) standards code, with the first two digits corresponding to a state code and the last three digits corresponding to a county code.

The poverty rate (`povty`) is defined to be the percentage of individuals between the ages of 18 and 64 in a county who lived in poverty in the census year 2000. The socio-economic factors are:

- `ag`: % work in agricultural sector
- `manu`: % work in manufacture sector
- `retail`: % work in retail sector
- `foreign`: % foreign born
- `feemp`: % of female employment
- `hsch`: % completed high school education
- `black`: % Black
- `hisp`: % Hispanic

We next load the dataset into a dataframe called `povdf` using the R function `read.table()`. To include variable names, we specify `header=TRUE`. Since we are loading a comma-separated file (`csv`), we also specify `sep=","`. We check the number of rows in the data frame using the function `nrow()` and the structure of the data frame using the `str()` function.

```
setwd("E:\\Postdoctoral Research Associate - UW Madison\\Teaching\\2026 Spring STAT679\\Data")
povdf <- read.table("uspoverty2000.csv", header = TRUE, sep = ",")

nrow(povdf)
```

```
## [1] 3070
```

```
str(povdf)
```

```
## 'data.frame':    3070 obs. of  12 variables:
##  $ FIPS_N  : int  1001 1003 1005 1007 1009 1011 1013 1015 1017 1019 ...
##  $ ag      : num  2.13 1.57 3.97 3.3 2.18 ...
##  $ black   : num  17.11 10.29 46.32 22.2 1.19 ...
##  $ feemp   : num  53.2 50.5 43.5 44.3 48.1 ...
##  $ foreign : num  1.17 2.11 1.5 0.43 3.1 ...
##  $ hisp    : num  1.4 1.76 1.65 1.01 5.33 ...
##  $ hsch    : num  33.8 29.6 32.4 35.7 36 ...
##  $ manu    : num  16.5 12.5 31.4 23.8 19.5 ...
##  $ povty   : num  9.06 9.13 21.91 17.74 9.89 ...
##  $ retail  : num  12.85 14.2 10.51 9.93 11.61 ...
##  $ STATEFP : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ COUNTYFP: int  1 3 5 7 9 11 13 15 17 19 ...
```

The data frame `povdf` features the poverty rate and the socio-economic variables from the census year 2000 in the counties of the 48 continental states. In order to look at Wisconsin and Illinois, we will subset the dataset.

The 2-digit FIPS code for Wisconsin is 55 and for Illinois is 17. We will split the variable `FIPS_n` into its first 2-digit code (state code) and call that new variable `FIPS_state`. The latter part of the FIPS code (county code) will be called `FIPS_county`.

We will use the `separate()` function from the `tidyr` R package to create a new dataframe called `povdf_sep`. This function takes its first argument the names of the variables we want to separate. Here, we want to split the variable `FIPS_n` into two new variables `into = c("FIPS_state", "FIPS_county")`. We specify that we want to separate after the first two digits using `sep = 2`.

The syntax below can be read as the data frame `povdf` is filtered into the `separate()` function. The function creates the new variables, named `FIPS_state` and `FIPS_county`. However, to save the new variables in a data frame, we must assign the results to an object. Below, we assign the results to a new data frame, `povdf_sep`. We check this step using the `str()` function.

```
povdf_sep <- povdf %>% separate(FIPS_N, into = c("FIPS_state", "FIPS_county"), sep = 2)

str(povdf_sep)
```

```
## 'data.frame':    3070 obs. of  13 variables:
##  $ FIPS_state : chr  "10" "10" "10" "10" ...
##  $ FIPS_county: chr  "01" "03" "05" "07" ...
##  $ ag         : num  2.13 1.57 3.97 3.3 2.18 ...
##  $ black      : num  17.11 10.29 46.32 22.2 1.19 ...
##  $ feemp      : num  53.2 50.5 43.5 44.3 48.1 ...
##  $ foreign    : num  1.17 2.11 1.5 0.43 3.1 ...
##  $ hisp       : num  1.4 1.76 1.65 1.01 5.33 ...
##  $ hsch       : num  33.8 29.6 32.4 35.7 36 ...
##  $ manu       : num  16.5 12.5 31.4 23.8 19.5 ...
##  $ povty      : num  9.06 9.13 21.91 17.74 9.89 ...
##  $ retail     : num  12.85 14.2 10.51 9.93 11.61 ...
```

```
##  $ STATEFP    : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ COUNTYFP   : int  1 3 5 7 9 11 13 15 17 19 ...
```

Having created a new variable `FIPS_state`, we subset the data frame to only include observations where `FIPS_state` equals either 55 (Wisconsin) or 17 (Illinois).

We use the `subset()` function to create a new data frame called `ilwipov`. The first argument is the data frame, `povdf_sep`. The second argument is the subsetting step. We want to keep observations where the variable `FIPS_state` is equal to `"55"` or (the vertical bar, `"|"`) where `FIPS_state` is equal to `"17"`. We include quotes around the numbers `"55"` and `"17"` because they are character types, which we know from the output of the `str()` function above.

```
ilwipov <- subset(povdf_sep, (FIPS_state == "55" | FIPS_state == "17"))

str(ilwipov)
```

```
## 'data.frame':    174 obs. of  13 variables:
##  $ FIPS_state : chr  "17" "17" "17" "17" ...
##  $ FIPS_county: chr  "001" "003" "005" "007" ...
##  $ ag         : num  2.83 2.61 4.77 1.72 8.94 ...
##  $ black      : num  3.07 34.9 7.41 0.9 18.2 ...
##  $ feemp      : num  57.2 41 54.2 58.8 58.6 ...
##  $ foreign    : num  0.81 0.88 0.81 7.45 1.64 ...
##  $ hisp       : num  0.83 1.44 1.43 12.49 3.93 ...
##  $ hsch       : num  37.5 34.7 34.2 38.2 28.8 ...
##  $ manu       : num  16.6 15.6 18.8 31.1 11.2 ...
##  $ povty      : num  9.35 23.19 8.73 6.17 7.44 ...
##  $ retail     : num  13.77 9.86 10.43 10.64 11.19 ...
##  $ STATEFP    : int  17 17 17 17 17 17 17 17 17 17 ...
##  $ COUNTYFP   : int  1 3 5 7 9 11 13 15 17 19 ...
```

```
head(ilwipov)
```

```
##     FIPS_state FIPS_county   ag black feemp foreign  hisp  hsch  manu povty
## 559         17         001 2.83  3.07 57.16    0.81  0.83 37.51 16.57  9.35
## 560         17         003 2.61 34.90 40.97    0.88  1.44 34.70 15.63 23.19
## 561         17         005 4.77  7.41 54.17    0.81  1.43 34.19 18.79  8.73
## 562         17         007 1.72  0.90 58.83    7.45 12.49 38.25 31.06  6.17
## 563         17         009 8.94 18.20 58.60    1.64  3.93 28.82 11.23  7.44
## 564         17         011 5.52  0.33 56.11    2.64  4.88 39.00 20.18  6.29
##     retail STATEFP COUNTYFP
## 559  13.77      17        1
## 560   9.86      17        3
## 561  10.43      17        5
## 562  10.64      17        7
## 563  11.19      17        9
## 564  12.81      17       11
```

To perform exploratory data analysis, we use summary statistics and graphical methods.

Using the R function `summary()`, we obtain several univariate summary statistics for each of the variables in the data frame `ilwipov`.

```
summary(ilwipov)
```

```
##   FIPS_state        FIPS_county              ag              black
##  Length:174         Length:174         Min.   : 0.080   Min.   : 0.0400
##  Class :character   Class :character   1st Qu.: 2.405   1st Qu.: 0.2125
```

```
##  Mode  :character   Mode  :character   Median : 4.500   Median : 0.5250
##                                        Mean   : 4.663   Mean   : 3.0794
##                                        3rd Qu.: 6.008   3rd Qu.: 3.6975
##                                        Max.   :16.310   Max.   :34.9000
##      feemp          foreign            hisp              hsch
##  Min.   :40.97   Min.   : 0.230   Min.   : 0.1800   Min.   :20.60
##  1st Qu.:52.70   1st Qu.: 0.810   1st Qu.: 0.7525   1st Qu.:34.21
##  Median :56.28   Median : 1.300   Median : 1.0950   Median :38.33
##  Mean   :56.41   Mean   : 2.164   Mean   : 2.3174   Mean   :37.52
##  3rd Qu.:60.16   3rd Qu.: 2.257   3rd Qu.: 2.6100   3rd Qu.:41.61
##  Max.   :70.33   Max.   :19.800   Max.   :23.7400   Max.   :47.53
##       manu           povty            retail         STATEFP
##  Min.   : 4.26   Min.   : 2.200   Min.   : 4.90   Min.   :17.00
##  1st Qu.:15.40   1st Qu.: 6.245   1st Qu.:10.60   1st Qu.:17.00
##  Median :19.18   Median : 8.090   Median :11.57   Median :17.00
##  Mean   :19.69   Mean   : 8.876   Mean   :11.66   Mean   :32.72
##  3rd Qu.:24.50   3rd Qu.:10.838   3rd Qu.:12.43   3rd Qu.:55.00
##  Max.   :38.28   Max.   :28.190   Max.   :23.70   Max.   :55.00
##     COUNTYFP
##  Min.   :  1.00
##  1st Qu.: 43.50
##  Median : 86.00
##  Mean   : 89.21
##  3rd Qu.:129.00
##  Max.   :203.00
```

For a continuous variable, the summary statistics are its minimum, first quartile, median, mean, third quartile, and maximum.

The summary statistics of the response variable of poverty rate show that the lowest county-level poverty rate was 2.200% and the highest county-level poverty rate was 28.190% among the 174 counties across Wisconsin and Illinois in 2000. The center of the county-level poverty rates is 8.090% measured by median and 8.876% measured by mean (i.e., average). In addition, the interquartile range is between the first quartile of 6.245% and the third quartile of 10.838%. That is, half of the counties had a poverty rate below 8.090% and the other half had a poverty rate above 8.090%, while the middle half of the counties had a poverty rate between 6.245% and 10.838%.

Visualization

We explore the relationship between the poverty rate and the socio-economic factors visually, by using the `ggplot2` package to create scatter plots.

We use the function `ggplot()` to create scatter plots. On the x-axis, we plot each of the 8 socio-economic factors and on the y-axis `povty`. We add additional features such as specifying opacity of the points with the argument `alpha` and setting it to 0.3 and setting the size of each point to 1 using `geom_point()`. We also add a localized smoothing line with the addition of `geom_smooth()` and specify the thickness of the smoothing line with the argument `size = 0.5`. We specify the height of the plot to be the same as the width using `theme(aspect.ratio = 1)`. Lastly, we add a title to each plot using `ggtitle()` and omit the grey background in the plot area with `theme_bw()`.

We first create all eight plots and save them as objects `p1` through `p8`, one plot for each socio-economic factor.

```
p1 <- ggplot(ilwipov, aes(x = ag, y = povty)) +
  geom_point(alpha = 0.3, size = 1) +
  geom_smooth(size = 0.5) +
  ggtitle("% AG") +
```

```
  theme(aspect.ratio = 1) +
  theme_bw()

p2 <- ggplot(ilwipov, aes(x = retail, y = povty)) +
  geom_point(alpha = 0.3, size = 1) +
  geom_smooth(size = 0.5) +
  ggtitle("% RETAIL") +
  theme(aspect.ratio = 1) +
  theme_bw()

p3 <- ggplot(ilwipov, aes(x = manu, y = povty)) +
  geom_point(alpha = 0.3, size = 1) +
  geom_smooth(size = 0.5) +
  ggtitle("% MANU") +
  theme(aspect.ratio = 1) +
  theme_bw()

p4 <- ggplot(ilwipov, aes(x = feemp, y = povty)) +
  geom_point(alpha = 0.3, size = 1) +
  geom_smooth(size = 0.5) +
  ggtitle("% FEEMP") +
  theme(aspect.ratio = 1) +
  theme_bw()

p5 <- ggplot(ilwipov, aes(x = foreign, y = povty)) +
  geom_point(alpha = 0.3, size = 1) +
  geom_smooth(size = 0.5) +
  ggtitle("% FOREIGN") +
  theme(aspect.ratio = 1) +
  theme_bw()

p6 <- ggplot(ilwipov, aes(x = hsch, y = povty)) +
  geom_point(alpha = 0.3, size = 1) +
  geom_smooth(size = 0.5) +
  ggtitle("% HSCH") +
  theme(aspect.ratio = 1) +
  theme_bw()

p7 <- ggplot(ilwipov, aes(x = black, y = povty)) +
  geom_point(alpha = 0.3, size = 1) +
  geom_smooth(size = 0.5) +
  ggtitle("% BLACK") +
  theme(aspect.ratio = 1) +
  theme_bw()

p8 <- ggplot(ilwipov, aes(x = hisp, y = povty)) +
  geom_point(alpha = 0.3, size = 1) +
  geom_smooth(size = 0.5) +
  ggtitle("% HISP") +
  theme(aspect.ratio = 1) +
  theme_bw()
```

Having created the 8 seperate scatter plots, we combine them all in to one figure using the function grid.arrange() from the package gridExtra. We enter the scatter plots to combine, p1 through p8, and specify

the number of columns the figure will have using the argument, ncol =4. We add an overall title to the bottom of the plot using the optional argument bottom.

```
grid.arrange(
  p1, p2, p3, p4, p5, p6, p7, p8,
  ncol = 4,
  bottom = "Figure 1: Poverty by Socio-Economic Factors in Wisconsin/Illinois in 2000"
)
```

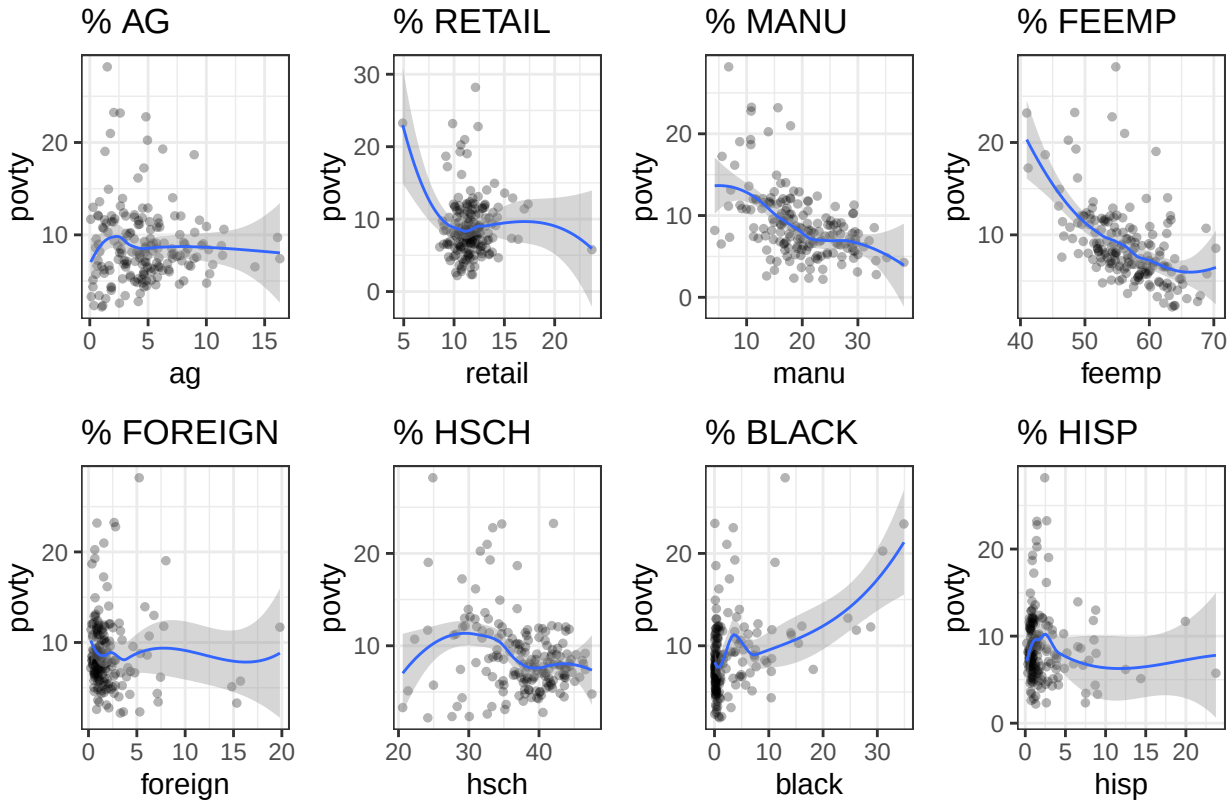


Figure 1: Poverty by Socio−Economic Factors in Wisconsin/Illinois in 2000

As female employment increases, there appears to be a steady decrease in poverty. The bulk of female employment is in the 50-60% range. Lowest female employment rates (approximately 40%) have high poverty rates (approximately 20%) and so on.